AdaptIQ

API Documentation

Complete reference for the AdaptIQ Adaptive Diagnostic Engine REST API — an AI-powered adaptive testing system built with FastAPI and MongoDB.

Version	**1.0.0**
Base URL	**http://localhost:8000/api/v1**
Protocol	**HTTP/1.1 · REST · JSON**
Auth	**None (session-based via session_id)**
Stack	Python · FastAPI · MongoDB · Groq AI
Repository	**github.com/Roshanchokhani/AdaptIQ**

Table of Contents

01	Overview & Architecture	Section 1
02	Data Models (MongoDB Schemas)	Section 2
03	Session Endpoints	Section 3
04	Question Endpoints	Section 4
05	Health Endpoints	Section 5
06	Error Reference	Section 6
07	Adaptive Algorithm Flow	Section 7

1 Overview & Architecture

AdaptIQ is a **1-Dimension Adaptive Testing** REST API that dynamically selects GRE-style questions based on a student's estimated ability level using **Item Response Theory (IRT)**. All responses are JSON. All request bodies use `Content-Type: application/json`.

> **Base URL:** `http://localhost:8000/api/v1`
> All endpoints below are relative to this base URL.

Module Structure

```
# Clean separation of concerns
app/
├── api/v1/endpoints/   # HTTP layer — routes only
│   ├── sessions.py     # POST /sessions, GET /next, POST /answer
│   └── questions.py    # Question CRUD + seed
├── services/           # Business logic
│   ├── session_service.py    # Adaptive loop orchestration
│   ├── question_service.py   # DB queries
│   └── insight_service.py    # Groq/Gemini/Anthropic AI
└── utils/              # Pure math, no I/O
    ├── irt.py                # 1PL Rasch model
    └── question_selector.py  # b-matching selector
```

2 Data Models (MongoDB Schemas)

Two collections are used. Indexes are created automatically on startup.

Collection: `questions`

```
{
  "_id":            ObjectId,
  "text":           string,
  "options": {
    "A": string,
    "B": string,
    "C": string,
    "D": string
  },
  "correct_answer": "A"|"B"|"C"|"D",
  "difficulty":     float // 0.1 – 1.0,
  "topic":          string,
  "tags":           string[],
  "explanation":    string
}
// Index: (difficulty, topic)
```

Collection: `user_sessions`

```
{
  "_id":           ObjectId,
  "user_id":       string,
  "current_ability": float,
  "attempts": [{
    "question_id":   string,
    "difficulty":    float,
    "topic":         string,
    "is_correct":    boolean,
    "ability_before": float,
    "ability_after":  float,
    "timestamp":     datetime
  }],
  "answered_question_ids": string[],
  "is_complete":   boolean,
  "study_plan":    object | null,
  "started_at":    datetime,
  "completed_at":  datetime | null
}
// Index: user_id, is_complete
```

3 Session Endpoints

Core adaptive testing flow. A student creates a session, fetches questions, and submits answers in a loop.

POST `/sessions/` — Create a new adaptive test session

Creates a new session for a student. The student begins with ability θ = 0.5 (baseline). Returns a `session_id` used in all subsequent calls.

Request Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`user_id`	`string`	required	Unique identifier for the student

REQUEST

```
POST /api/v1/sessions/
Content-Type: application/json

{
  "user_id": "student_001"
}
```

RESPONSE — 201 CREATED

```
{
  "session_id": "64f3a2b1c9d4e5f6a7b8c9d0",
  "message":    "Session created. Call GET /next to begin."
}
```

`201 Created` `422 Validation Error`

`GET` **/sessions/{session_id}/next** — Get the next adaptive question

Returns the most informative next question for the student's current ability level using **b-matching** — selects the unanswered question whose difficulty is closest to the student's current θ. The `correct_answer` field is intentionally omitted.

Path Parameter

FIELD	TYPE	DESCRIPTION
`session_id`	`string`	The session ObjectId returned from POST /sessions/

REQUEST

```
GET /api/v1/sessions/64f3a2b1c9d4e5f6a7b8c9d0/next
```

RESPONSE — 200 OK

```
{
  "id":         "64f3a2b1c9d4e5f6a7b8c9d1",
  "text":       "If x² - 5x + 6 = 0, what are the values of x?",
  "options": {
    "A": "2 and 3",
    "B": "1 and 6",
    "C": "-2 and -3",
    "D": "3 and 4"
  },
  "difficulty": 0.45,
  "topic":      "Quantitative Reasoning",
  "tags":       ["algebra", "quadratic", "factoring"]
}
```

> **Note:** `correct_answer` is never included in this response to prevent client-side cheating. It is only returned after the student submits their answer via POST /answer.

`200 OK` `404 Session Not Found` `404 No Questions Available` `409 Session Complete`

`POST` `/sessions/{session_id}/answer` Submit an answer and update ability score

Processes the student's answer, updates the ability score θ using the **1PL Rasch IRT gradient**, and persists the attempt atomically. After every 10 answers, an AI-generated personalized study plan is created and returned.

Path Parameter

FIELD	TYPE	DESCRIPTION
session_id	string	The session ObjectId

Request Body

FIELD	TYPE	REQUIRED	DESCRIPTION
question_id	string	required	The question ObjectId from GET /next
answer	string	required	Student's answer: "A", "B", "C", or "D"

REQUEST

```
POST /api/v1/sessions/64f3a2b1c9d4e5f6a7b8c9d0/answer
Content-Type: application/json

{
  "question_id": "64f3a2b1c9d4e5f6a7b8c9d1",
  "answer":      "A"
}
```

RESPONSE — 200 OK (QUESTIONS 1–9)

```
{
  "is_correct":        true,
  "correct_answer":    "A",
  "explanation":       "Factor as (x-2)(x-3)=0, giving x=2 and x=3.",
  "ability_before":    0.50,
  "ability_after":     0.63,
  "questions_answered": 5,
  "session_complete":  false,
  "study_plan":        null
}
```

RESPONSE — 200 OK (QUESTION 10 — INCLUDES AI STUDY PLAN)

```
{
  "is_correct":        false,
  "correct_answer":    "B",
  "explanation":       "...",
  "ability_before":    0.72,
  "ability_after":     0.65,
  "questions_answered": 10,
  "session_complete":  false,
```

```
  "study_plan": {
    "assessment": "Student shows strong quantitative skills but struggles with verbal
reasoning.",
    "steps": [
      {
        "step": 1,
        "focus":         "Verbal Reasoning",
        "action":        "Complete 30 GRE vocabulary flashcards daily.",
        "resource_type": "flashcards"
      },
      {
        "step": 2,
        "focus":         "Reading Comprehension",
        "action":        "Read one academic article and summarize the argument.",
        "resource_type": "reading"
      },
      {
        "step": 3,
        "focus":         "Analytical Writing",
        "action":        "Watch 3 videos on GRE essay structure.",
        "resource_type": "video"
      }
    ]
  }
}
```

200 OK | 400 Already Answered | 404 Session Not Found | 404 Question Not Found | 409 Session Complete

---

GET `/sessions/{session_id}/status` — Get full session state

Returns the complete session document including all attempts, current ability score, completion status, and the AI study plan (if generated).

RESPONSE — 200 OK

```
{
  "_id":              "64f3a2b1c9d4e5f6a7b8c9d0",
  "user_id":          "student_001",
  "current_ability":  0.65,
  "attempts":         [ /* array of QuestionAttempt objects */ ],
  "answered_question_ids": ["..."],
  "is_complete":      false,
  "study_plan":       null,
  "started_at":       "2025-01-15T10:30:00Z",
  "completed_at":     null
}
```

200 OK | 404 Session Not Found

4 Question Endpoints

Admin endpoints for managing the question bank. Used for seeding and inspection.

POST `/questions/seed` — Bulk-seed 22 GRE questions from JSON

Loads all questions from `data/seed_questions.json` and inserts them into MongoDB. Safe to call multiple times — skips if questions already exist.

RESPONSE — 201 CREATED

```
{
  "message": "Successfully seeded 22 questions."
}
```

`201 Created`

GET `/questions/` — List all questions with optional filters

Query Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`topic`	`string`	optional	Filter by topic name (e.g. "Quantitative Reasoning")
`min_difficulty`	`float`	optional	Minimum difficulty (0.1–1.0)
`max_difficulty`	`float`	optional	Maximum difficulty (0.1–1.0)

EXAMPLE REQUEST

```
GET /api/v1/questions/?topic=Quantitative+Reasoning&min_difficulty=0.4&max_difficulty=0.7
```

`200 OK`

`POST` **/questions/** — Create a single question

Request Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`text`	`string`	required	Question text
`options`	`object`	required	Map of A/B/C/D to answer text
`correct_answer`	`string`	required	"A", "B", "C", or "D"
`difficulty`	`float`	required	IRT b-parameter: 0.1–1.0
`topic`	`string`	required	e.g. "Quantitative Reasoning"
`tags`	`string[]`	required	e.g. ["algebra", "quadratic"]
`explanation`	`string`	required	Shown to student after answering

`201 Created` `422 Validation Error`

`GET` **/questions/{question_id}** — Get a question by ID

Returns the full question document including `correct_answer`.

`200 OK` `404 Not Found`

`DELETE` **/questions/{question_id}** — Delete a question by ID

RESPONSE — 200 OK

```
{
  "message": "Question '64f3...' deleted."
}
```

`200 OK` `404 Not Found`

5 Health Endpoints

GET `/health` — API health check

RESPONSE — 200 OK

```
{ "status": "healthy" }
```

6 Error Reference

All errors return a JSON body with a `detail` field.

HTTP CODE	ERROR	CAUSE
400	Already Answered	Submitting the same question_id twice in a session
404	Session Not Found	Invalid or expired session_id
404	Question Not Found	Invalid question_id in answer submission
404	No Questions Available	All questions answered, none remaining
409	Session Complete	Attempting to get a question or answer after session ends
422	Validation Error	Missing required fields or invalid types in request body

ERROR RESPONSE FORMAT

```
{
  "detail": "Session '64f3...' is already complete."
}
```

7 Adaptive Algorithm Flow

The adaptive loop uses the **1-Parameter Logistic (Rasch) IRT model**. Each call to POST /answer executes this sequence:

```
Student answers question (difficulty b, current ability θ) ↓ P = 1 / (1 + exp(-(θ - b))) // probability of correct given θ, b ↓ gradient = response - P // 1 if correct, 0 if wrong ↓ θ_new = clamp(θ + 0.3 × gradient, 0.1, 1.0) ↓ If questions_answered == 10 → call Groq AI → generate StudyPlan ↓ next_question = argmin |q.difficulty - θ_new| // b-matching ↓ Correct answer → gradient > 0 → θ increases → harder question Incorrect answer → gradient < 0 → θ decreases → easier question
```

Why IRT and not a simple ±0.1 jump?
The gradient magnitude is largest when P ≈ 0.5 (well-targeted item) and smallest when the question is too easy or too hard. This means the ability estimate self-corrects naturally — a student getting a very easy question wrong causes a small θ drop, not a large one.

Complete Test Lifecycle

EVENT	TRIGGER	RESULT
Session created	POST /sessions/	θ = 0.5, session active
Question delivered	GET /next	b-matched question, no correct_answer
Answer submitted	POST /answer	θ updated, next question selected
AI Study Plan	10th answer submitted	Groq generates 3-step plan, stored in session
Session complete	20th answer submitted	is_complete = true, completed_at set

Endpoint Quick Reference

METHOD	ENDPOINT	PURPOSE
POST	/api/v1/sessions/	Start a new adaptive test
GET	/api/v1/sessions/{id}/next	Get next adaptive question
POST	/api/v1/sessions/{id}/answer	Submit answer, get feedback + θ update
GET	/api/v1/sessions/{id}/status	Get full session state
POST	/api/v1/questions/seed	Seed 22 GRE questions
GET	/api/v1/questions/	List questions (filterable)
POST	/api/v1/questions/	Create a question
GET	/api/v1/questions/{id}	Get question by ID

METHOD	ENDPOINT	PURPOSE
DELETE	`/api/v1/questions/{id}`	Delete a question
GET	`/health`	Health check